AI-POWERED CONTEXT-AWARE BIDIRECTIONAL INDIAN SIGN LANGUAGE TRANSLATOR WITH MULTILINGUAL VOICE ASSISTANCE AND EMERGENCY ALERT SYSTEM

A PROJECT REPORT

Submitted by

HARIHARAN A — 2303601710421019

JAIMANI R — 2303601710421020

KAMALESH P — 2303601710421022

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

GOVERNMENT COLLEGE OF ENGINEERING, BARGUR – 635 104

ANNA UNIVERSITY :: CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report entitled "AI-POWERED CONTEXT-AWARE BIDIRECTIONAL INDIAN SIGN LANGUAGE TRANSLATOR WITH MULTILINGUAL VOICE ASSISTANCE AND EMERGENCY ALERT SYSTEM" is the bonafide work of HARIHARAN A (2303601710421019), JAIMANI R (2303601710421020) and KAMALESH P (2303601710421022), who carried out the project work under my supervision in partial fulfillment for the award of the degree of Bachelor of Engineering in Computer Science and Engineering.

SIGNATURE	**SIGNATURE**
Dr. J. Nafeesa Begum **HEAD OF THE DEPARTMENT** Department of Computer Science and Engineering Government College of Engineering, Bargur – 635 104	Mrs. G. Dhivya **SUPERVISOR** Assistant Professor (Temp) Department of Computer Science and Engineering Government College of Engineering, Bargur – 635 104

Submitted for the project viva-voce examination held on: _____

Internal Examiner: _____ External Examiner: _____

ABSTRACT

This project presents SignOra, a bidirectional Indian Sign Language (ISL) communication application combining camera-based gesture recognition, text-to-sign presentation, multilingual voice assistance and a configurable emergency-alert workflow. The application provides separate Sign-to-Text and Text-to-Sign pages.

Sign-to-Text uses MediaPipe landmarks with a 63-feature static classifier for 61 labels and a 40-frame BiGRU using 225 features per frame for 262 word classes. Predictions pass confidence and temporal-consensus gates before transcript entry. Text-to-Sign selects available word clips or fingerspells unmatched words. Translation and speech features depend on browser, operating-system and network support.

Stored test artifacts report 97.01% static-model accuracy (34,333 samples), 81.33% BiGRU accuracy (857 samples) and 75.61% emergency-model accuracy (82 samples). These datasets are not directly comparable. The eight-class emergency model is experimental, not a validated safety device; alert delivery depends on external configuration and connectivity. This report presents the design, implementation, evaluation evidence and limitations.

Keywords: Indian Sign Language, gesture recognition, MediaPipe, BiGRU, multilingual speech, text-to-sign, emergency alert.

TABLE OF CONTENTS

ABSTRACT iii
LIST OF TABLES vi
LIST OF FIGURES vii
LIST OF ABBREVIATIONS viii
CHAPTER 1 INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Significance and Scope 2
1.5 Project Contributions 2
1.6 Report Organization 3
CHAPTER 2 LITERATURE SURVEY 4
2.1 Literature Survey 4
2.2 Summary of Existing Research 4
2.2.1 Static and Temporal Classifiers 4
2.2.2 Bidirectional Communication and Speech 5
2.3 Research Gap 5
2.4 Existing System and Emergency Notification 6
CHAPTER 3 PROPOSED SYSTEM AND IMPLEMENTATION 7
3.1 Proposed System 7
3.1.1 Feasibility Analysis 8
3.2 System Architecture 8
3.3 System Workflow 8
3.3.1 System Data Flow 8
3.3.2 Use Scenarios 9
3.4 Modules and User Interface 10
3.4.1 Static Landmark Model 11

3.4.2 Temporal BiGRU Model 12
3.4.3 Optional LSTM and Emergency Recognition 13
3.4.4 Prediction Stability and Transcript 14
3.4.5 Text-to-Sign and Language Assistance 15
3.4.6 Emergency Webhook Integration 16
3.5 Dataset and Feature Representation 19
3.6 Algorithms and Prediction 19
3.7 Software and Hardware Requirements 19
3.8 Detailed Design and Testing 19
CHAPTER 4 CONCLUSION AND FUTURE ENHANCEMENTS 21
4.1 Result Analysis and Discussion 21
4.1.1 Static Model Metrics 21
4.1.2 BiGRU and Emergency Results 22
4.1.3 Software Test Coverage 23
4.1.4 Demonstration Evidence 24
4.2 Conclusion 24
4.3 Future Enhancements 25
CHAPTER 5 APPENDICES 26
5.1 Local Setup and Operation 26
5.1.1 Setup 26
5.1.2 User Workflow 26
5.2 Model and Data Summary 26
5.3 Safety, Privacy and Deployment Notes 27
5.3.1 Responsible Use 27
CHAPTER 6 REFERENCES 28

LIST OF TABLES

TABLE NO. NAME OF THE TABLE

Table 3.1: Functional requirements 7
Table 3.2: Non-functional requirements and constraints 7
Table 3.3: Major modules and responsibilities 9
Table 3.4: Representative application endpoints 10
Table 4.1: Recorded held-out model evaluation 21
Table 4.2: Verification cases and expected behavior 24
Table 5.1: Bundled model pathways 26

LIST OF FIGURES

FIGURE NO. NAME OF THE FIGURE

Figure 3.1: High-level SignOra application and emergency-alert architecture 8

Figure 3.2: Sign-to-Text interface in light theme 11

Figure 3.3: Static landmark classifier recognizing the letter L 11

Figure 3.4: BiGRU sequence buffering while 40 frames are collected ... 13

Figure 3.5: BiGRU prediction for the word Mouse 13

Figure 3.6: Emergency model collecting a 30-frame sequence 14

Figure 3.7: Experimental emergency-sign recognition and alert workflow 14

Figure 3.8: Text-to-Sign phrase entry and sequence player 16

Figure 3.9: Text-to-Sign sequence displayed for a Tamil phrase 16

Figure 3.10: Manual emergency request with user confirmation 18

Figure 3.11: Successful response from the configured emergency endpoint 18

Figure 3.12: Example n8n workflow connecting a webhook to Twilio ... 18

Figure 4.1: Static classifier confusion matrix from the saved evaluation artifact 22

Figure 4.2: BiGRU training accuracy history from recorded artifacts 23

Figure 4.3: Emergency model accuracy history from recorded artifacts . 23

LIST OF ABBREVIATIONS

Abbreviation	Expansion / meaning
API	Application Programming Interface
BiGRU	Bidirectional Gated Recurrent Unit
CNN	Convolutional Neural Network; project label for the static landmark classifier
FPS	Frames per second
HTTP / HTTPS	Hypertext Transfer Protocol / Secure HTTP
ISL	Indian Sign Language
LSTM	Long Short-Term Memory
ML	Machine Learning
NLP	Natural Language Processing
SOS	Emergency distress alert
TTS	Text-to-Speech
UI	User Interface
UTC	Coordinated Universal Time

CHAPTER 1
INTRODUCTION

1.1 Background

Communication is essential for education, healthcare, public services and everyday participation. Deaf and hard-of-hearing people who communicate through sign language may encounter barriers when an interlocutor does not understand the language. An assistive system should support more than a single recognition model: it should provide readable text from signed input, a reverse text-to-sign path, speech options and a way to request help.

SignOra is a browser-based project application for Indian Sign Language. It combines computer vision and landmark-based machine learning with a Flask service and web UI. Its two user-facing pages are Sign to Text and Text to Sign; auxiliary services handle translation, speech, model status and a configurable emergency webhook.

1.2 Problem Statement

A single-frame recognizer may show unstable labels, repeat a held sign or commit an uncertain prediction. Static alphabet recognition alone cannot represent a continuous sentence. A useful prototype therefore needs confidence filtering, temporal buffering, a transcript and a reverse text-to-sign path.

Emergency communication introduces additional failure modes: a classifier can be wrong, and network delivery can fail. The application must not imply that an experimental model guarantees assistance. SignOra gates predictions, confirms manual alerts, keeps webhook credentials server-side and reports delivery status.

1.3 Objectives

The aim is to implement an accessible prototype for bidirectional ISL communication, multilingual voice assistance and configurable emergency notifications.

Objectives are to recognize selected static and dynamic gestures; stabilize live predictions before transcript entry; provide reverse text-to-sign using word clips and fingerspelling; support speech and translation; expose model and translation functions through a web service; protect a configurable emergency webhook; and evaluate available models using stored evaluation artifacts and software tests.

1.4 Significance and Scope

The system recognizes only labels represented by loaded models. The static model has 61 classes, while the configured target list contains 81. The BiGRU supports 262 word classes. Text-to-sign coverage depends on available clips; unmatched words are fingerspelled. Translation requires internet access and speech behavior depends on browser and operating-system capabilities.

Performance depends on framing, lighting, signer, device and model generalization. The emergency model has eight labels and limited test data. It is not a certified interpreter, medical product or safety system.

1.5 Project Contributions

The implementation integrates multiple model families behind a common live prediction experience, confidence-based temporal consensus, duplicate-commit control, transcript tools, a reverse sign-sequence player, speech and language functions, and a protected

webhook workflow. Available evaluation artifacts and screenshots support a transparent account of demonstrated behavior.

1.6 Report Organization

Chapter 1 introduces the problem and scope. Chapter 2 reviews technical background. Chapter 3 specifies requirements and constraints. Chapter 4 presents system design. Chapter 5 describes implementation. Chapter 6 gives verification and evaluation evidence. Chapter 7 concludes and lists future work. Appendices summarize setup, model pathways and responsible-use notes.

CHAPTER 2
LITERATURE SURVEY

2.1 Literature Survey

Sign recognition maps visual motion and hand configuration to linguistic labels. Relevant evidence may be spatial (hand shape and position), temporal (motion across frames) or contextual (sequence and meaning). The system must tolerate changes in signer, background, angle and illumination. Recognition of isolated alphabetic signs is narrower than translation of continuous, grammatical sign language.

2.2 Summary of Existing Research

Landmark-based approaches represent a hand or body using keypoints rather than classifying every pixel. MediaPipe hand tracking provides 21 landmarks; three coordinates per landmark yield 63 scalar values. This compact representation simplifies downstream classification and explicit normalization, but depends on reliable detection and may fail under occlusion.

The BiGRU pathway uses MediaPipe Holistic landmarks: 33 pose points plus two sets of 21 hand points, each with three coordinates, giving 225 features per frame.

2.2.1 Static and Temporal Classifiers

A static classifier is suitable for classes whose information is primarily contained in one hand pose. The repository's train_cnn.py accepts 63-feature vectors, standardizes them and defines fully connected Dense layers with batch normalization and dropout. Although the checkpoint is called CNN, this landmark architecture does not perform convolution over image pixels; this report makes that distinction explicit.

Dynamic signs require temporal evidence. LSTM and BiGRU networks process ordered feature vectors across frames. The bundled BiGRU receives 40 frames by 225 features. An optional LSTM pathway uses 30 hand-landmark frames if a matching checkpoint and labels are available. Sequence models may represent motion but require more data, preprocessing and inference time.

2.2.2 Bidirectional Communication and Speech

A two-way experience includes recognition from camera input and presentation of text as visual sign content. A clip-based pathway can show a whole-word video and fingerspell words without a clip. This supports more vocabulary but is not equivalent to fluent signed grammar, expression and spatial reference.

Speech input and output provide another channel. Translation can connect a supported source language to an English sign library, but may alter meaning and depends on network access. Browser speech APIs and installed voices vary by platform.

2.3 Research Gap

The project integrates selected functions into a demonstrable prototype rather than claiming a general-purpose ISL translator. It addresses unstable frame predictions, supports static and sequence model selection, provides word-clip/fingerspelling fallback and isolates emergency delivery behind a configured service. Vocabulary limits, external translation and experimental emergency performance remain explicit.

2.4 Existing System and Emergency Notification

An emergency workflow combines classification and message delivery, each with independent failure modes. Confidence thresholds and multi-frame agreement can reduce accidental activation but cannot eliminate confident errors. Server-side credentials, HTTPS, rate limiting and delivery status reduce avoidable exposure; they do not prove that an external responder received or acted on a notification.

CHAPTER 3

PROPOSED SYSTEM AND IMPLEMENTATION

3.1 Proposed System

A single-purpose gesture recognizer leaves translation, reading and speech to separate tools. A manual alert disconnected from recognition also forces the user to switch workflows. SignOra integrates recognition, transcript handling, text-to-sign playback, multilingual speech/translation and webhook notification. It is not a replacement for a qualified interpreter or certified emergency channel.

Table 3.1: Functional requirements

ID	Requirement	Implementation evidence
FR-01	Display Sign-to-Text UI and receive camera frames.	Camera stream and frame-prediction routes.
FR-02	Support available static and sequence models.	Model selector and readiness reporting.
FR-03	Filter low-confidence and unstable predictions.	Confidence threshold, consensus and commit cooldown.
FR-04	Build, clear and update a transcript.	Sentence APIs and transcript controls.
FR-05	Convert typed text into sign sequence.	Video clips and fingerspelling fallback.
FR-06	Support speech and configured voice languages.	Browser speech input, synthesis and translation.
FR-07	Send manual or recognized-sign alert.	Emergency API and server-side webhook.
FR-08	Expose health and model status.	Health, model-info and performance routes.

Table 3.2: Non-functional requirements and constraints

Category	Requirement / constraint
Usability	Clear mode selection, status, transcript feedback and explicit alert confirmation.
Reliability	Reject unavailable artifacts; require confidence and temporal agreement.
Security	Keep webhook URL/token server-side, require HTTPS and rate-limit requests.
Performance	Interactive inference depends on hardware, model and frame cadence.
Compatibility	Use compatible Python, TensorFlow/Keras and browser features.
Privacy	Camera/transcript handling requires consent and deployment controls.
Safety	Emergency classification is experimental, not life-critical assurance.

3.1.1 Feasibility Analysis

Technical feasibility is supported by the model artifacts, Python packages and browser UI. Operation requires a webcam, sufficient compute, browser camera permission and optional network services. Open-source components reduce prototype cost, while hosting, external APIs and inference hardware may add deployment costs. Responsible use requires consent, retention controls, accessibility testing and clear limitations.

3.2 System Architecture

The system comprises a browser UI, Flask application, recognition and language utilities, model artifacts, sign-video resources and an optional external webhook. The backend handles frame prediction, model readiness, transcript state, translation, speech and alert dispatch. An incomplete model is not offered as ready.

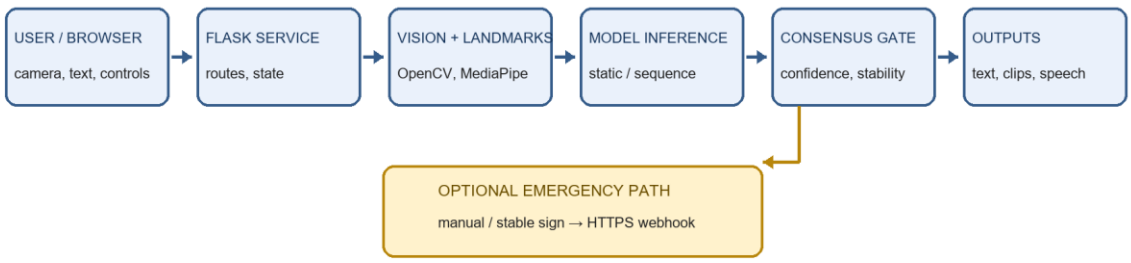


Figure 3.1: High-level SignOra application and emergency-alert architecture

3.3 System Workflow

3.3.1 System Data Flow

For recognition, a frame is acquired and converted for the vision pipeline. MediaPipe produces landmarks; temporal models build fixed-length sequences. The classifier returns a class and score. A smoother rejects invalid, weak or unstable candidates. A stable prediction is

committed once, then the transcript and UI are updated. No-hand input resets pending candidates and supports re-arming.

For Text to Sign, input is translated into English where required, tokenized and mapped to available word clips. Unsupported words use letter clips. Playback controls allow step, repeat and speed changes.

For alerts, the browser confirms manual requests or submits a stable emergency classification. The server creates a JSON event and posts it to the configured HTTPS webhook. Per-client limits and server-only credentials apply; failures are reported without forwarding the remote response body.

3.3.2 Use Scenarios

For Sign to Text, a user grants camera permission, selects a ready model, presents a gesture and reviews the transcript after stable predictions. For Text to Sign, a user enters or speaks a phrase, reviews any English preview and plays the generated sequence. For emergency support, the user confirms a manual request or the model recognizes a supported sign; the service sends a configured JSON event and returns delivery status.

Table 3.3: Major modules and responsibilities

Module	Responsibility
templates/index.html	Sign-to-Text page, status and transcript.
templates/text_to_sign.html	Phrase entry and sign-sequence playback.
app.py	Routes, model state, transcript, translation/speech and alerts.
src/models/gesture_recognizer.py	Model loading and static/sequence inference.
src/preprocessing/preprocess_data.py	Landmark preprocessing and normalization.
src/utils/prediction_smoothing.py	Consensus and duplicate-commit gating.
src/utils/nlp_processor.py	Gesture-sequence and text-processing helpers.
src/utils/tts_handler.py	Speech synthesis and queued playback.
models/ and public/text-to-sign/	Weights, labels, statistics and sign videos.

Table 3.4: Representative application endpoints

Endpoint	Purpose
/api/predict_frame	Submit a frame for gesture inference.
/api/models	Report available model readiness.
/api/translate	Translate a text phrase.
/api/speech-audio and /api/speak	Generate or play speech output.
/api/emergency-alert and /api/emergency-sos	Request alert dispatch.
/api/gesture_data and /api/update_sentence	Read or update transcript state.
/health and /api/performance	Expose health and runtime information.

3.4 Modules and User Interface

Sign to Text presents camera input, a model selector, recognition status and transcript controls. Only successfully loaded models appear as ready. Text to Sign accepts a phrase and displays ordered clips with playback and stepping controls. Screenshots demonstrate light/dark themes; vocabulary remains limited by installed assets.

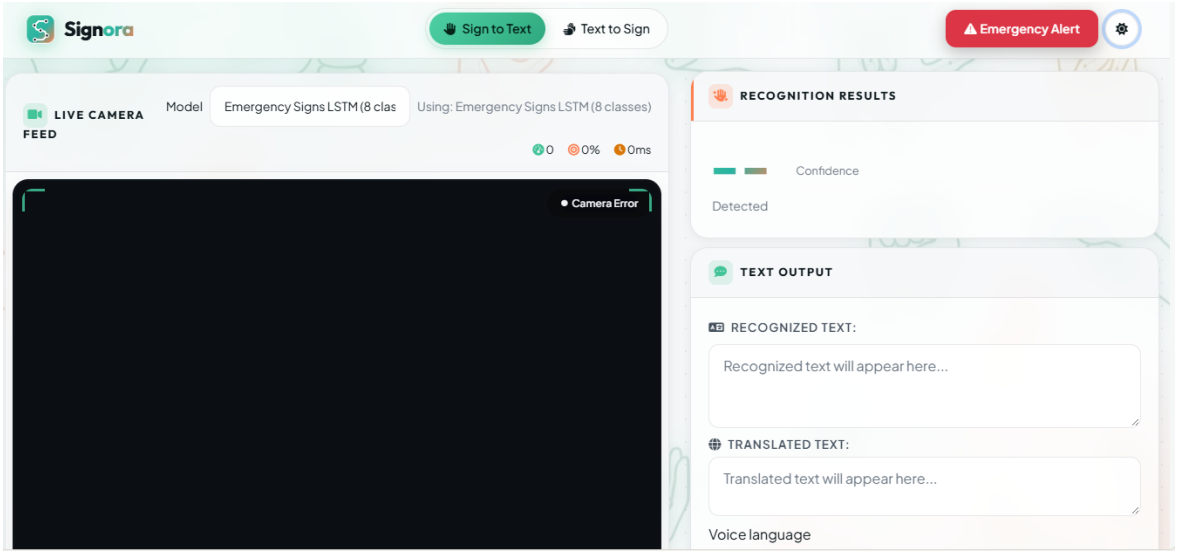


Figure 3.2: Sign-to-Text interface in light theme

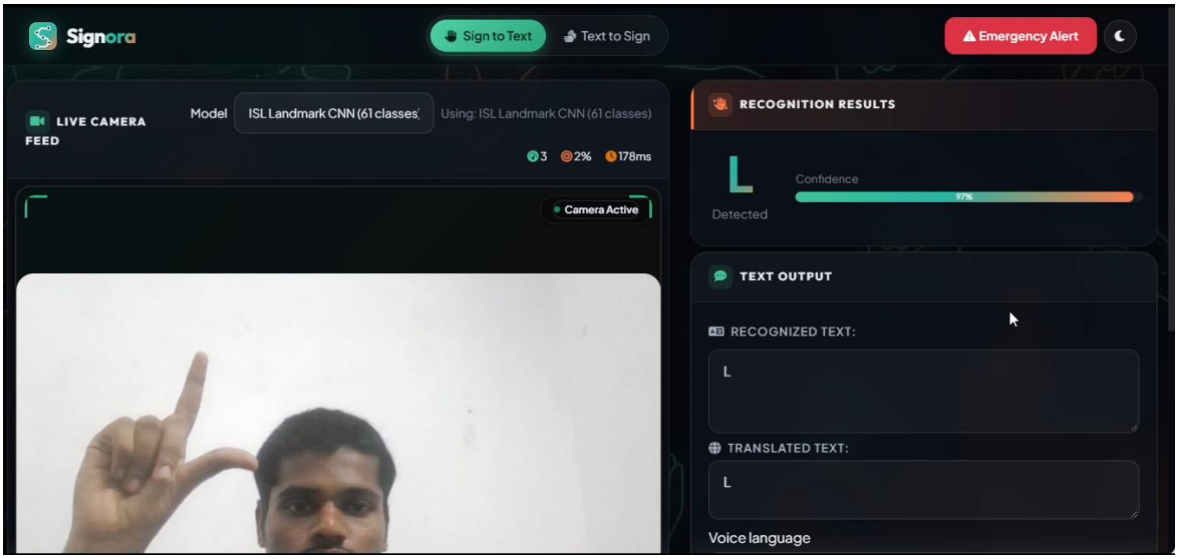


Figure 3.3: Static landmark classifier recognizing the letter L

3.4.1 Static Landmark Model

The static model receives 21 hand landmarks with x, y and z coordinates (63 values). Training preprocessing translates landmarks relative to the wrist and scales by maximum landmark distance. A standard scaler fitted during training is reused for inference. The bundled label mapping and scaler are checked against model dimensions.

The train_cnn.py script defines fully connected Dense layers, batch normalization and dropout. The repository uses CNN as a saved model

identifier, but the current landmark network does not convolve image pixels.

3.4.2 Temporal BiGRU Model

MediaPipe Holistic yields a 225-feature frame vector: pose 33×3, left hand 21×3 and right hand 21×3. The model consumes 40 frames, applies training-time aspect correction and normalization, and predicts 262 classes. Until 40 frames are collected, the UI reports buffering. The loader checks model, labels and normalization artifacts.

The dataset summary records 4,284 videos and 262 classes, with 857 test samples. A short missing-hand grace period supports brief tracking interruptions; latency depends on the device.

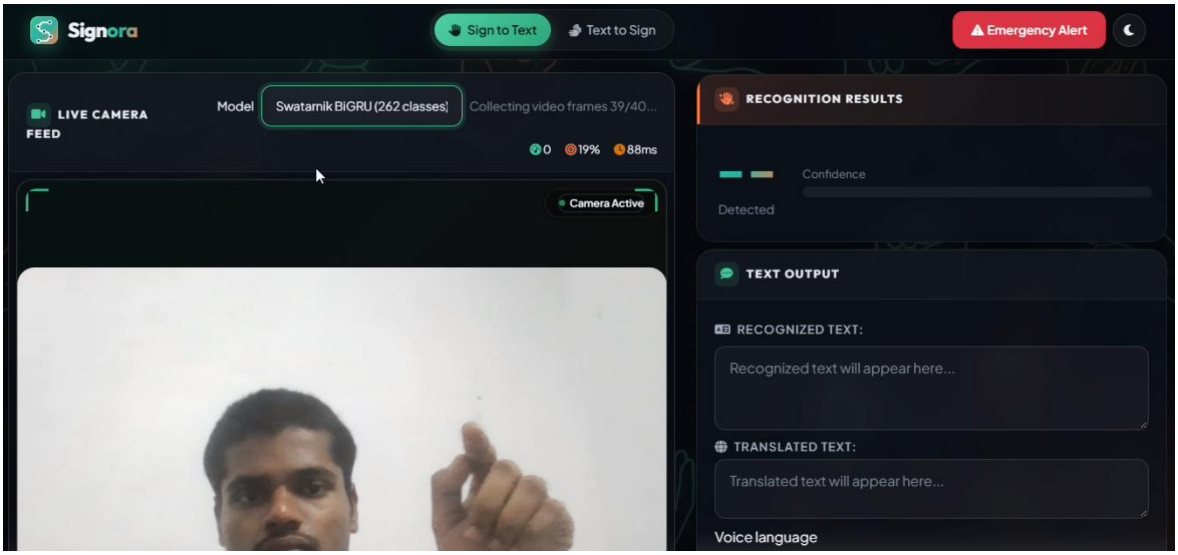


Figure 3.4: BiGRU sequence buffering while 40 frames are collected

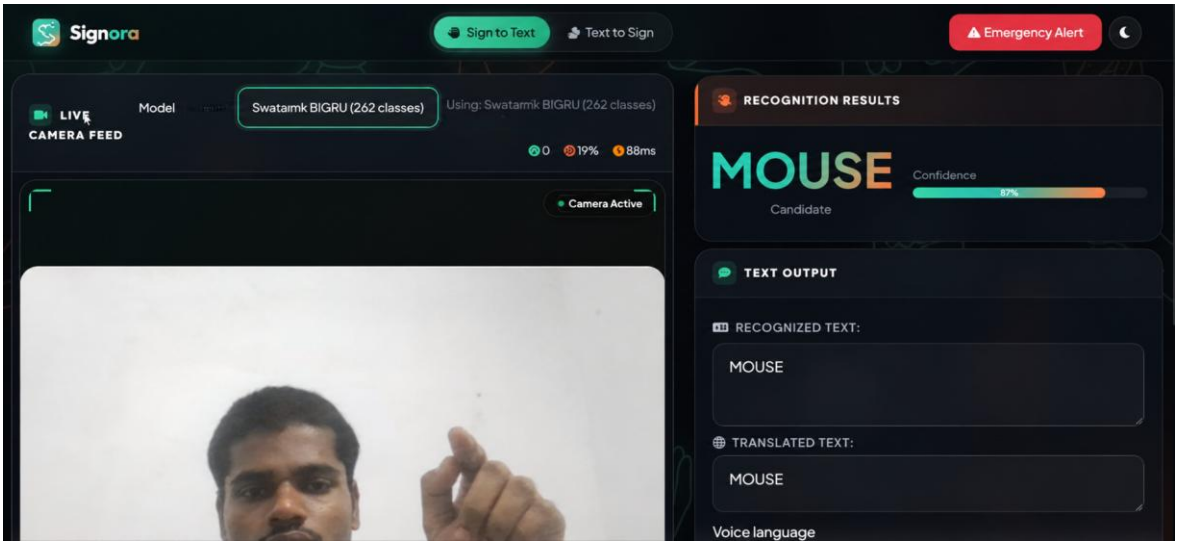


Figure 3.5: BiGRU prediction for the word Mouse

3.4.3 Optional LSTM and Emergency Recognition

The optional Dynamic LSTM loader requires a trained 30-frame hand-landmark checkpoint and matching labels. The emergency pathway is a separate 30-frame classifier with eight labels: accident, call, doctor, help, hot, lose, pain and thief. Its score must reach 0.65 before temporal consensus. This reduces uncertain triggers but cannot eliminate confident errors.

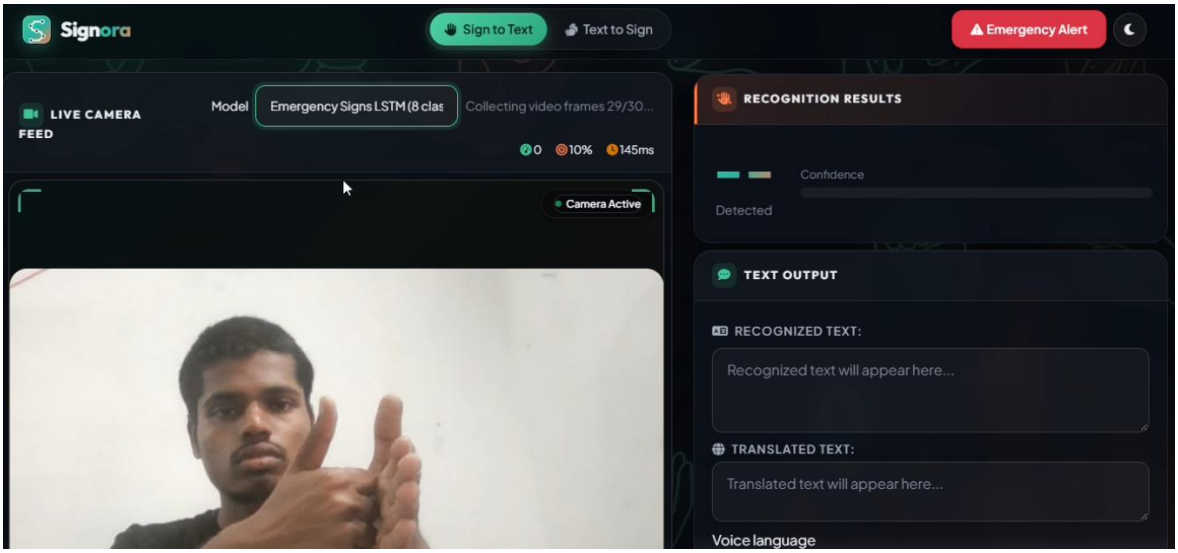


Figure 3.6: Emergency model collecting a 30-frame sequence

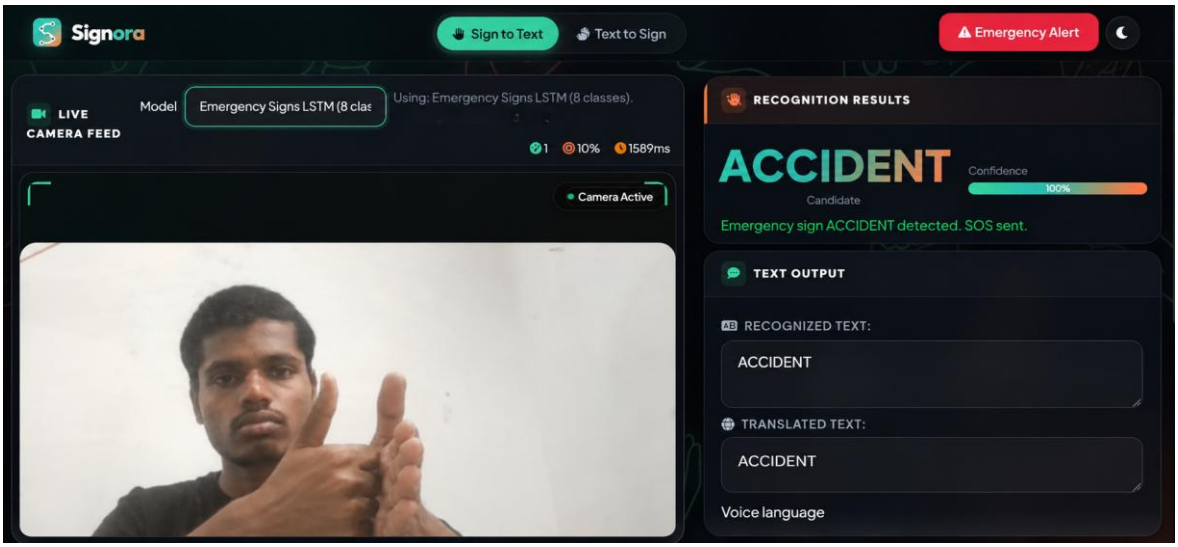


Figure 3.7: Experimental emergency-sign recognition and alert workflow

3.4.4 Prediction Stability and Transcript

The smoother stores a bounded window of recent labels. A candidate must exceed 0.60 confidence and have at least three stable observations in a five-frame window. Invalid predictions, absent hands and low confidence reset the window. A committer applies cooldowns and blocks repeat commits for a held sign until release. These rules improve interaction but cannot correct a confident model error.

The interface can show buffering or an unconfirmed candidate without appending it to the sentence. Transcript state can be cleared or updated through application APIs.

3.4.5 Text-to-Sign and Language Assistance

Text-to-Sign maps phrases to available sign videos. Whole-word clips are preferred; letter clips provide a fingerspelling fallback. Non-English input can be translated to English for the available sign library, with an English preview. Speech input uses browser recognition where available. Eight voice languages are exposed, subject to browser, OS and network support.

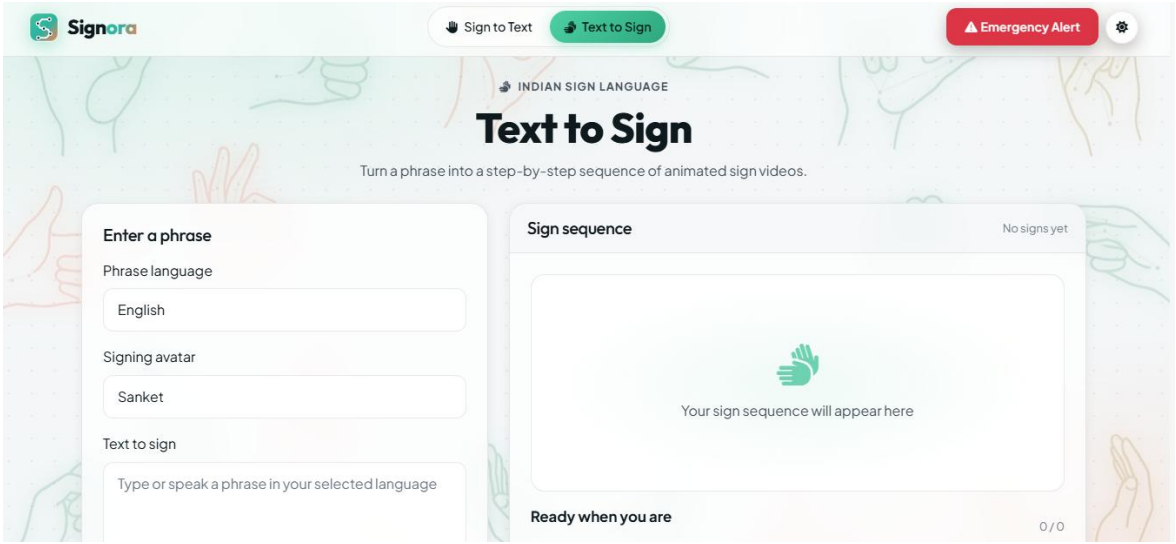


Figure 3.8: Text-to-Sign phrase entry and sequence player

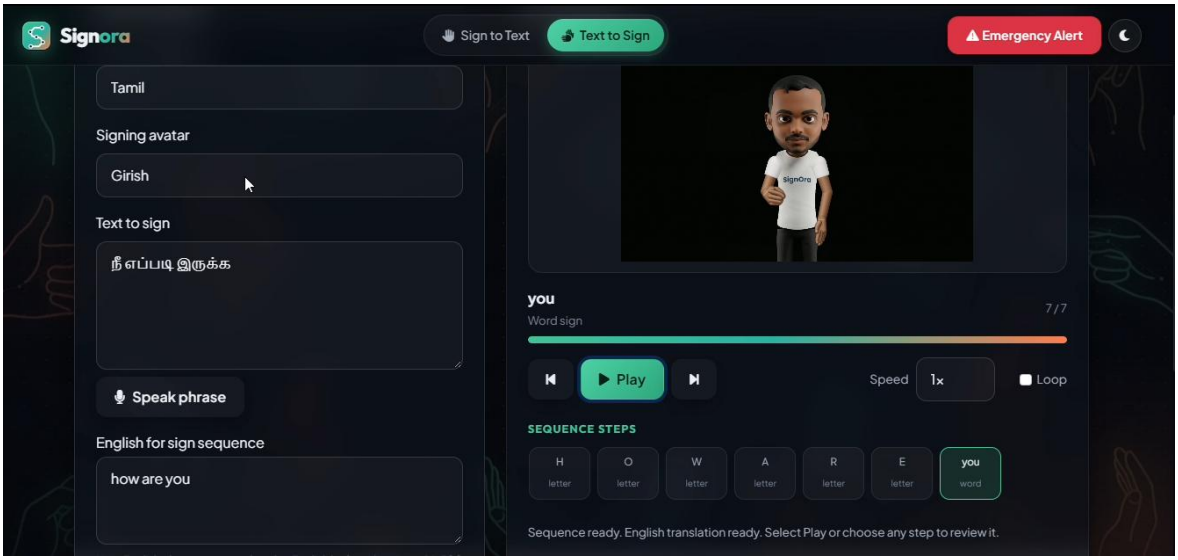


Figure 3.9: Text-to-Sign sequence displayed for a Tamil phrase

3.4.6 Emergency Webhook Integration

The emergency endpoint supports a manual request and can be invoked by a recognized emergency sign. The server posts an event, source, message and UTC timestamp. WEBHOOK_URL and WEBHOOK_TOKEN are configured outside browser code; HTTPS is required and one attempt per client IP per minute is allowed. Non-2xx responses are reported without disclosing the remote response body.

The supplied screenshots demonstrate an n8n/Twilio workflow. Those services are deployment choices, not bundled guarantees. Credentials, workflow activation, network and recipient configuration are required.

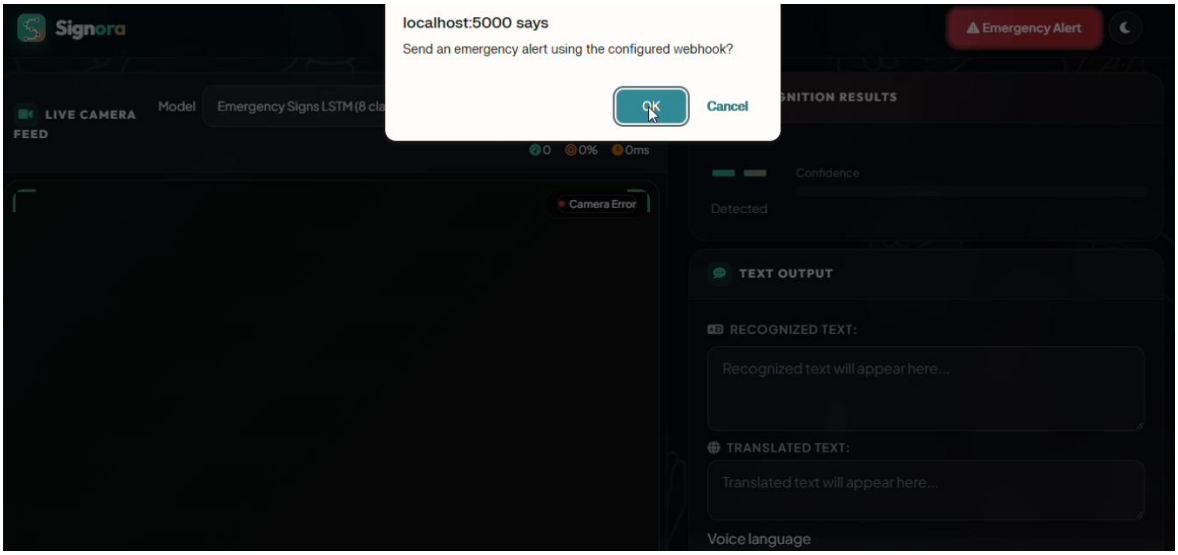


Figure 3.10: Manual emergency request with user confirmation

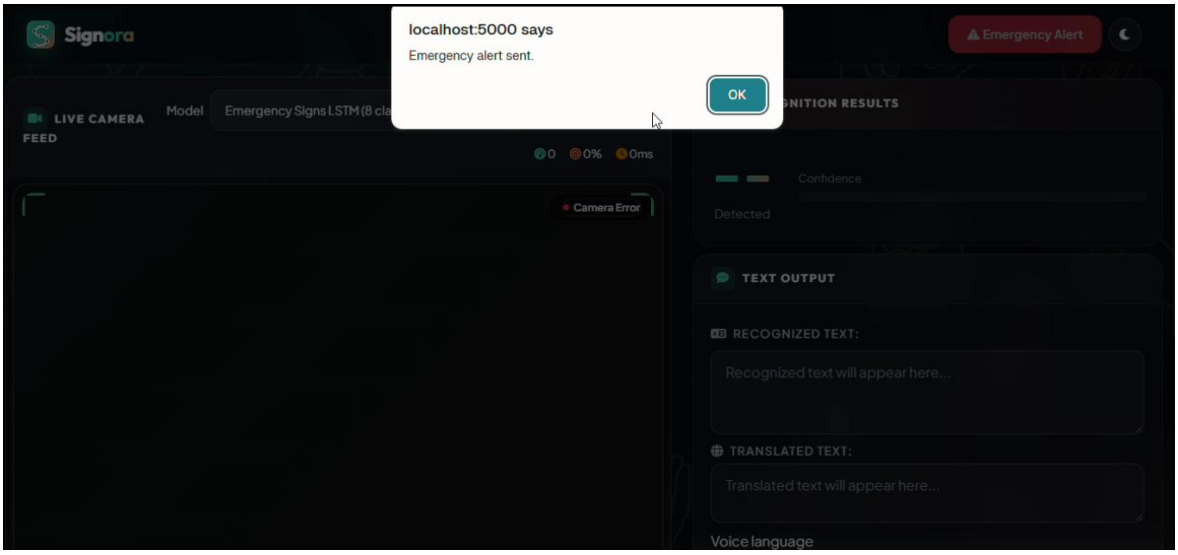


Figure 3.11: Successful response from the configured emergency endpoint

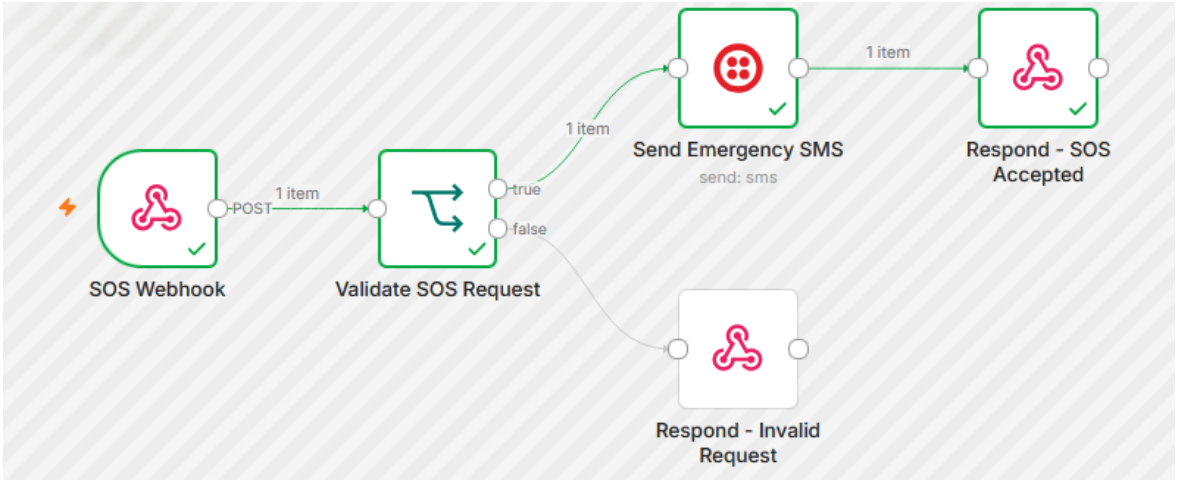


Figure 3.12: Example n8n workflow connecting a webhook to Twilio

3.5 Dataset and Feature Representation

The static classifier uses normalized 63-value hand-landmark samples; its saved confusion-matrix evaluation represents 34,333 test samples. The BiGRU dataset contains 4,284 videos across 262 classes, with 857 held-out test sequences. The emergency dataset contains 408 clips across eight classes, split into 260 training, 66 validation and 82 test samples. These datasets have different vocabularies and collection protocols and are evaluated independently.

3.6 Algorithms and Prediction

The static checkpoint uses a dense neural classifier over standardized hand landmarks, although the saved project identifier uses the name CNN. Dynamic recognition uses recurrent sequence models: a 40-frame BiGRU and optional 30-frame LSTM pathways. Live labels are filtered using confidence thresholds and temporal consensus before transcript or alert actions.

3.7 Software and Hardware Requirements

The stack includes Python, Flask, Flask-CORS, OpenCV, MediaPipe, TensorFlow/Keras, NumPy and scikit-learn, with browser technologies for the UI. Runtime hardware should include a camera, adequate memory and a current browser. Translation and external notifications require internet access; speech support varies.

3.8 Detailed Design and Testing

Training and inference share the same landmark normalization and feature dimensions. Model loaders verify checkpoint inputs, output-label mappings and required normalization files. Automated tests exercise these contracts, temporal smoothing, no-hand resets, emergency

thresholds and brief tracking loss. No unimplemented joint-angle or exercise formulas are assumed for this sign-language project.

CHAPTER 4
CONCLUSION AND FUTURE ENHANCEMENTS

4.1 Result Analysis and Discussion

Repository tests cover model-to-label dimensional consistency, matching training and inference normalization, temporal smoothing, emergency confidence gating, BiGRU input shape and brief tracking loss. Screenshots show UI flows but do not replace accessibility, compatibility or field testing.

Metrics below come from saved project evaluation artifacts. Static-model results represent 34,333 samples in its confusion matrix; BiGRU and emergency tests contain 857 and 82 samples respectively. Different datasets, class sets and protocols mean the accuracy values are not direct comparative benchmarks.

Table 4.1: Recorded held-out model evaluation

Model	Classes / input	Test n	Accuracy	Macro F1	Interpretation
Static landmark classifier (CNN identifier)	61 / 63 features	34,333	97.01%	95.23%	Recorded artifact.
BiGRU + Holistic landmarks	262 / 40×225	857	81.33%	80.64%	Notebook/report metrics.
Emergency LSTM	8 / 30-frame sequence	82	75.61%	73.59%	Experimental; not safety validated.

4.1.1 Static Model Metrics

The static model reports 97.01% accuracy, 99.83% top-k accuracy, macro F1 of 0.9523 and weighted F1 of 0.9699. These scores apply to the stored evaluation set and do not establish performance for unseen users, cameras or conditions, nor continuous sentence translation.

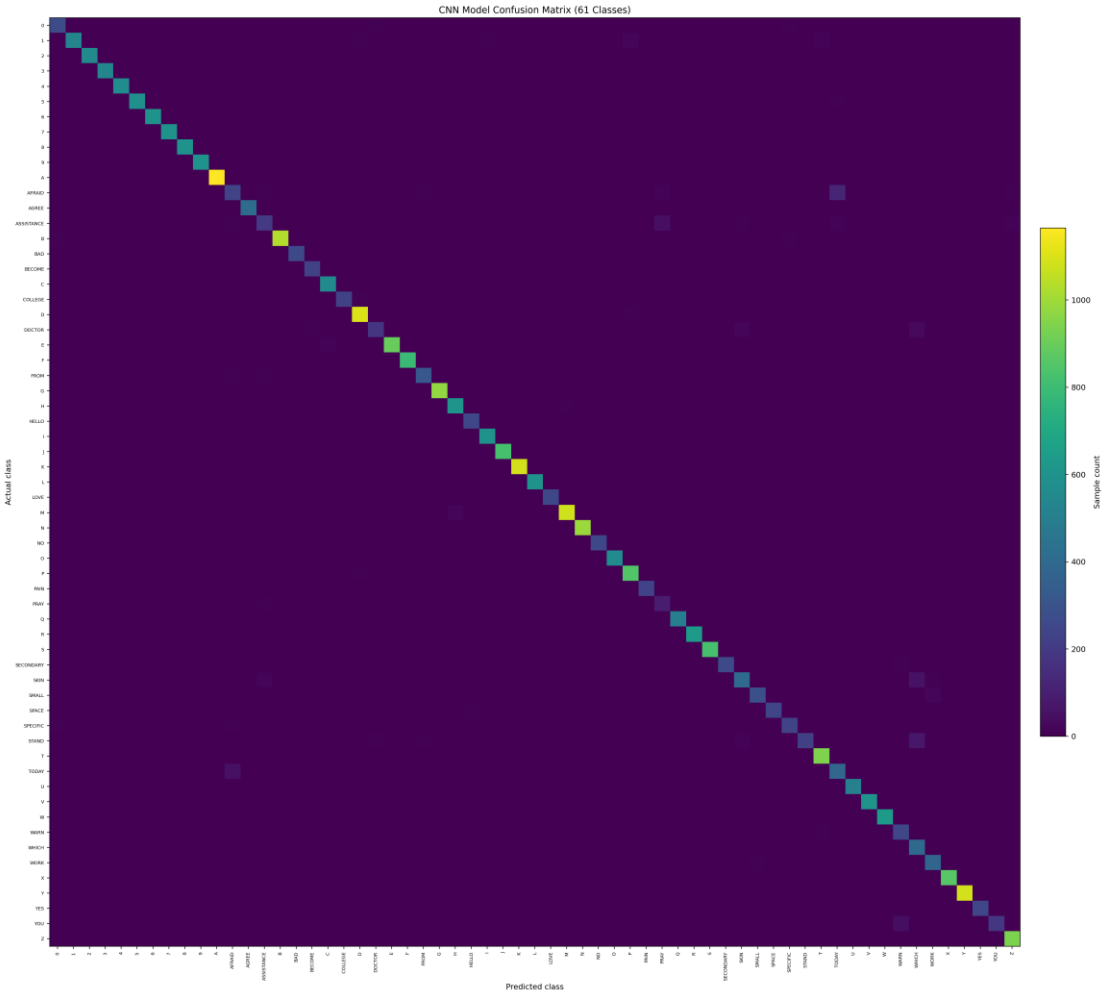


Figure 4.1: Static classifier confusion matrix from the saved evaluation artifact

4.1.2 BiGRU and Emergency Results

The BiGRU reports 81.33% test accuracy and 0.8064 macro F1 over 262 classes and 857 samples. The dataset summary records 4,284 videos; signer diversity and capture conditions should be included in further generalization studies.

The emergency classifier reports 75.61% accuracy and 0.7359 macro F1 across eight labels using 82 test samples. Its limited test set and false-alert risks mean that it must not replace established emergency communication.

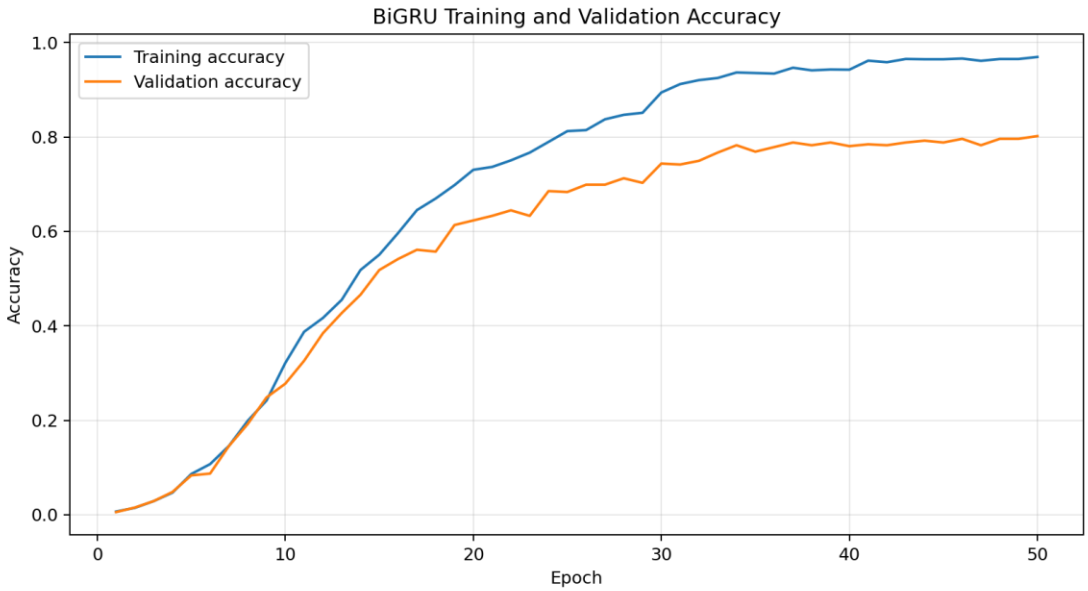


Figure 4.2: BiGRU training accuracy history from recorded artifacts

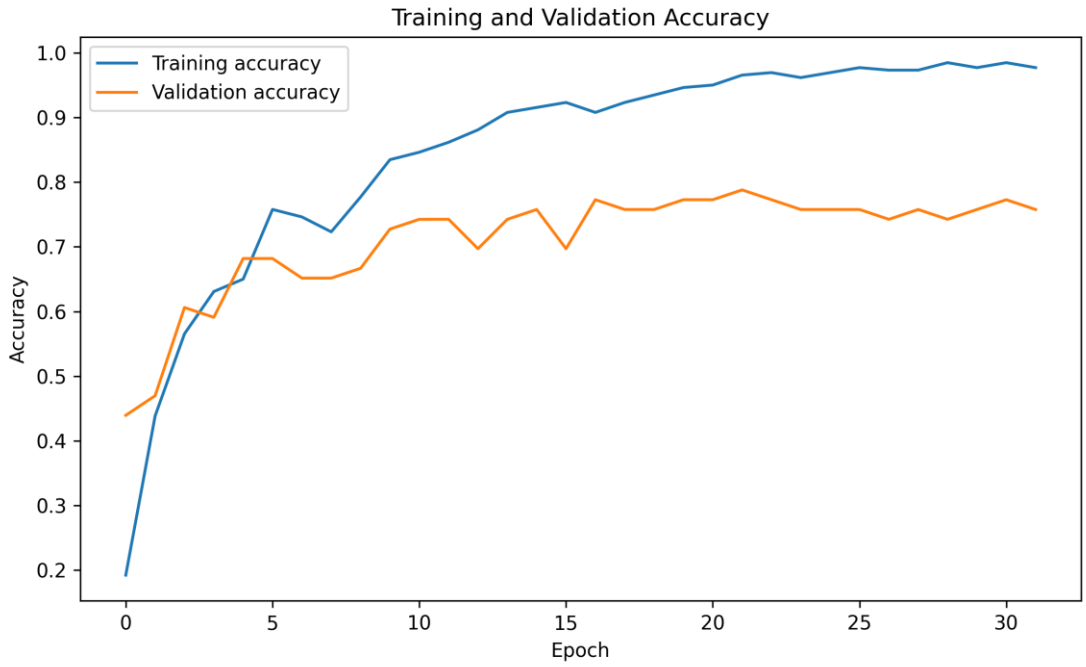


Figure 4.3: Emergency model accuracy history from recorded artifacts

4.1.3 Software Test Coverage

Automated checks assert that the static model accepts 63 features and emits 61 labels; the BiGRU accepts 40×225 input and maps 262 labels; and training/live preprocessing produces equivalent normalized vectors.

Other tests verify confidence consensus, no-hand reset, emergency thresholding and the BiGRU missing-hand grace period.

These tests verify selected implementation invariants, not real-world accuracy. Further testing should include multiple signers, held-out sessions, webcams, translation quality, browser microphones, webhook failures and intended-user studies.

Table 4.2: Verification cases and expected behavior

Test area	Expected result
Model shape and labels	Output dimensions agree with saved class mapping.
Landmark normalization	Training and inference feature vectors agree.
Temporal consensus	No commit before confident agreeing frame count.
No-hand reset	Pending candidates clear and signs can be re-armed.
Emergency gating	Scores below 0.65 do not enter temporal consensus.
Artifact readiness	Unavailable or mismatched models are not marked ready.
Webhook delivery	2xx is delivered; failures expose status, not response body.
Text-to-sign fallback	Unmatched words use available letter clips.

4.1.4 Demonstration Evidence

The supplied images show light/dark themes, static recognition, 40-frame BiGRU buffering and prediction, English and Tamil sign sequences, emergency sequence collection, automatic-alert examples, manual confirmation and a sample workflow. These are demonstrations, not independent acceptance testing.

4.2 Conclusion

SignOra demonstrates an integrated prototype for selected ISL gestures and text-to-sign presentation. It combines a 63-feature static landmark classifier, a 40-frame BiGRU, an optional LSTM path, prediction smoothing, transcript controls, multilingual speech/translation and a server-side emergency webhook.

Recorded metrics show strong static classification on its stored set, lower performance on the larger-vocabulary BiGRU task, and an experimental emergency model with limited test data. Vocabulary coverage, signer diversity, generalization and fail-safe behavior must improve before broader use. The application is not a certified interpreter or emergency safety system.

4.3 Future Enhancements

Collect a consented, balanced dataset across signers, sessions and recording conditions; publish reproducible splits and per-class metrics; and evaluate unseen-signer generalization. Describe the static classifier accurately and compare a true image CNN only under controlled protocols. Continuous ISL translation needs temporal language modeling, grammar, facial expression and non-manual cues.

Expand licensed sign clips, add phrase-level rendering and obtain fluent-signer review. Assess translation quality with language experts, and improve keyboard access, captions, screen-reader semantics and community usability testing. Add structured observability, retention controls, deployment security and external-service integration tests.

Keep emergency alerts separate from life-critical assurance. Further work requires extensive validation, false-positive/false-negative analysis, human-centered confirmation and reliable fallback channels. No alert model should be the only means of seeking help.

CHAPTER 5
APPENDICES

5.1 Local Setup and Operation

5.1.1 Setup

Use RUN_LOCALLY.md and the requirements files as the installation authority. Create or activate the project virtual environment, install the selected requirements, configure from .env.example and start the provided local runner. Do not commit secrets. Live inference requires a camera available to the process performing inference.

For emergency delivery, configure WEBHOOK_URL and WEBHOOK_TOKEN in an ignored .env file or deployment secrets. Use HTTPS and test the integration safely before enabling alerts. The endpoint must return 2xx for delivery to be reported as successful.

BiGRU inference needs compatible model, class mapping, task file and normalization statistics. Dynamic LSTM needs a trained checkpoint and matching labels. Select another ready model if an artifact is missing.

5.1.2 User Workflow

Sign to Text: allow camera access, select a ready model, hold a sign until consensus and review the transcript. Release the hand before repeating a held sign. Text to Sign: select phrase language, enter or speak a phrase, review the preview and use the player. Emergency Alert: confirm a manual request and review delivery status; use a trusted alternate channel if notification is unavailable.

5.2 Model and Data Summary

Table 5.1: Bundled model pathways

Pathway	Input	Vocabulary	Status / note

Pathway	Input	Vocabulary	Status / note
Static classifier	21 hand landmarks × 3 = 63	61	Checkpoint, labels and scaler required.
Swatarnik BiGRU	40 × 225 holistic features	262	Requires model, classes, task and stats.
Dynamic LSTM	30 hand-landmark frames	Checkpoint-dependent	Optional, requires trained artifacts.
Emergency LSTM	30-frame sequence	8	Experimental; gated and temporally filtered.

Retain model versions, class mappings and documented data splits with evaluation artifacts. Retraining can produce different results.

5.3 Safety, Privacy and Deployment Notes

5.3.1 Responsible Use

Users should be informed when a camera is active and what information may be sent. Credentials remain server-side and should be rotated if exposed. Production deployment requires HTTPS, appropriate access controls, rate limiting and a privacy/retention policy.

The emergency classifier is experimental. A successful webhook response confirms only that the configured endpoint accepted a request; it does not confirm that a responder received or acted on it.

CHAPTER 6

REFERENCES

[1] Flask Project (2026), “Flask Documentation,” Pallets Projects. Available: https://flask.palletsprojects.com/ (accessed 25 October 2026).

[2] Google for Developers (2026), “Hand Landmarker guide for Python,” MediaPipe documentation. Available: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker/python (accessed 25 October 2026).

[3] OpenCV (2026), “OpenCV-Python Tutorials,” OpenCV Documentation. Available: https://docs.opencv.org/ (accessed 25 October 2026).

[4] Python Software Foundation (2026), “Python 3 Documentation.” Available: https://docs.python.org/3/ (accessed 25 October 2026).

[5] SignOra project (2026), “AI-Powered Context-Aware Bidirectional Indian Sign Language Translator with Multilingual Voice Assistance and Emergency Alert System,” source modules, model artifacts, README and evaluation results supplied with this project.

[6] TensorFlow (2026), “Keras: The high-level API for TensorFlow,” TensorFlow Documentation. Available: https://www.tensorflow.org/guide/keras (accessed 25 October 2026).

[7] Twilio (2026), “Messaging documentation,” Twilio Docs. Available: https://www.twilio.com/docs/messaging (accessed 25 October 2026).